



24-PRIVACY-CONSENT-AND-LEGAL.md — Whose face is it, and who is liable?



O2 — The photographer owns the decision to share. AURA processes photographs of people at the most private moment of their lives. Under GDPR, faces processed for identification are **biometric data**: special category, highest protection. Get this wrong and you do not get a warning, you get a fine and a photographer whose business you damaged.

1. Who is who, legally

This is the distinction everything else hangs on, and it is the one most local-first apps get wrong.

Party	Role	Responsibility
The couple and guests	Data subjects	Have rights: access, erasure, objection
The photographer	Data controller	Decides why and how the images are processed; holds the model release; answers to the couple
You, when AURA runs locally	Neither — a tool vendor	You never receive the data. This is the whole point of offline-first, and it is your strongest legal position
You, when a cloud call is made	Data processor	Now you need a Data Processing Agreement with every photographer, and a sub-processor list
Anthropic / OpenAI / whoever	Sub-processor	Must be disclosed by name, with jurisdiction, before the first byte leaves

The strategic consequence: every feature you can keep on-device removes an entire legal apparatus. The three-tier compute pipeline is not only a performance decision — it is the reason you can sell to a German or a British wedding photographer at all.

2. The consent model — frozen contract

Consent is per project, never global, never a checkbox in Settings that someone ticked in 2026 and forgot.

```
// crates/aura-core/src/contract/consent.rs -- FROZEN CONTRACT
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
pub struct ProjectConsent {
```

```

pub project_uuid: Uuid,
/// Cloud reasoning on *metadata only* (EXIF, counts, timestamps).
/// No pixels. Default: false.
pub cloud_metadata: bool,
/// Cloud vision on *downscaled, face-blurred* frames. Default: false.
pub cloud_derived_imagery: bool,
/// Cloud vision on original imagery. Default: false.
/// Requires the photographer to type the project name to enable.
pub cloud_full_imagery: bool,
/// Anonymous product telemetry for this project. Default: false.
pub telemetry: bool,
/// Contribute cull decisions (no pixels) to model improvement.
pub improve_models: bool,
pub granted_at: Option<DateTime<Utc>>,
pub granted_by: String,
/// Consent expires; silence is not consent forever.
pub expires_at: Option<DateTime<Utc>>,
pub revoked_at: Option<DateTime<Utc>>,
}

impl Default for ProjectConsent {
    /// Every field false. There is no constructor that defaults to true.
    fn default() -> Self { /* all false, all None */ }
}

/// The ONLY way any byte reaches the network. Enforced by the
/// egress gate in 19-OBSERVABILITY-AND-FORENSICS.md.
pub trait ConsentGate {
    fn permits(&self, c: &ProjectConsent, payload: &EgressPayload)
        -> Result<Permit, AuraError>; // AURA-SEC-9001 on denial
}

```

Rules that follow from the type:

- Default deny. A `ProjectConsent::default()` permits nothing, so a forgotten initialisation fails closed.
- Consent is checked at the egress gate, not at the call site. A developer who forgets cannot bypass it; there is no other socket.
- Revocation is immediate and retroactive where possible: in-flight requests are cancelled, cached cloud responses for that project are purged.
- Every consent change writes to the decision ledger with a timestamp and a reason.

3. The four data classes and where each may travel

Class	Examples	Local disk	Logs	Telemetry	Cloud
C0 — Public	App version, OS, GPU model	Yes	Yes	Yes	Yes
C1 — Operational	Image count, durations, error codes	Yes	Yes	With consent	With consent
C2 — Identifying	File paths, filenames, folder names, camera serial	Yes	Hashed only	Never	Never
C3 — Biometric	Pixels of faces, face embeddings, identity clusters	Yes, encrypted	Never	Never	Only with explicit <code>cloud_full_imagery</code>

A filename is C2 because `2026-06-14_Sharma-Weds-Patel/bride-getting-ready-0042.CR3` names two real people and a date. Logging it is a data breach that looks like a debug line.

4. Face embeddings — the highest-risk artefact in the product

Face grouping is a headline feature and a legal landmine simultaneously.

- **Embeddings never leave the device.** Not for backup, not for sync, not for "improving the model", not ever. Put this in writing in the privacy policy, because it is a real differentiator and it is also a promise you can actually keep.
- **Encrypted at rest** with a key in the OS keychain (Windows DPAPI, macOS Keychain, Linux Secret Service). If the laptop is stolen, the embeddings are not readable.
- **Bound to the project.** Deleting a project deletes its embeddings in the same transaction. No orphan face database accumulating across every wedding a photographer ever shot.
- **No cross-project identity linking by default.** "This guest also appeared at a wedding in March" is technically easy, is genuinely useful for repeat clients, and is a surveillance feature. It is opt-in, per photographer, off by default, and clearly labelled.
- **Retention default: purge embeddings 90 days after project completion.** The grouping result (which photo belongs to which cluster) survives; the biometric template does not.

5. What the photographer must be able to tell their client

Ship this as a one-page PDF the photographer can hand over, generated from the project's actual consent state. It is a support tool and a sales tool at once.

- Which processing happened entirely on the photographer's computer (almost always: all of it).
- Whether anything was sent to a cloud service, which service, in which country, and what exactly was sent.

- That no image was used to train anyone's AI.
- How to request deletion, and that deletion is real and verifiable.
- That AURA never altered anyone's face, body or skin colour — which is a product invariant, not a policy.

6. Regional obligations, honestly scoped

Regime	What it actually demands of AURA
GDPR / UK GDPR	Biometric data is Article 9. Local-only processing keeps you out of controller status. The moment cloud imagery is enabled you need a DPA, a sub-processor list, an SCC for US transfers, and a documented lawful basis
Illinois BIPA	The strictest in the world; private right of action, per-scan statutory damages. Face embeddings created without written consent are enormous exposure. Local-only, opt-in, purged — which is exactly the design above
Texas CUBI, Washington MHMD	Similar shape, enforced by the state AG rather than by class action
CCPA / CPRA	Applies to you only above thresholds, but the "do not sell or share" signal is cheap to honour from day one
EU AI Act	AURA is minimal-risk: it does not identify strangers, score people, or make consequential decisions about anyone. Do not accidentally leave that category by adding "guest sentiment scoring" or attractiveness ranking
India DPDP, Nepal, Brazil LGPD	Consent-centric regimes; the per-project consent record satisfies the core requirement

Do not attempt to be your own lawyer for the DPA and privacy policy. Budget for a few hours of specialist review before the first paid customer. The engineering above is what makes that review short and cheap.

7. Features that must never be built

Write this list into `CLAUDE.md` so an eager agent never "helpfully" adds one:

- Identifying a person against any external or cross-customer database.
- Age, gender, ethnicity or attractiveness inference — in scoring, sorting, or as a hidden model input.
- Skin lightening, body reshaping, or any change to what a person looks like.
- Uploading imagery for "quality assurance" or "debugging" without per-project consent.
- A crash report that includes a thumbnail.
- Silent telemetry, or telemetry enabled by an update.
- Selling, sharing or aggregating any customer imagery or derived embedding.

8. Tests and gates

- ☐ **Egress gate test:** every network-capable module is called with `ProjectConsent::default()`; all must fail with `AURA-SEC-9001`.
- ☐ **Poisoned fixture test:** a project whose paths contain `SECRET-BRIDE-NAME`; run the full pipeline, then grep every log file, telemetry payload, crash dump and run bundle for that string. Any hit fails the build.
- ☐ **Embedding containment test:** run with a network sniffer asserting zero bytes of embedding data on any socket.
- ☐ **Revocation test:** revoke mid-run; assert in-flight requests cancel and the cloud cache for that project is purged.
- ☐ **Deletion test:** delete a project; assert embeddings, caches, previews, ledger rows and temp files are gone, verified by filesystem scan and by a SQLite integrity query.
- ☐ **Expiry test:** consent past `expires_at` behaves exactly as no consent.
- ☐ **Class-leak test:** an assertion in the telemetry serialiser that no C2 or C3 field can be encoded, enforced by the type system where possible.

9. Acceptance criteria

- ☐ `consent.rs` matches section 2 byte for byte and is under contract freeze.
- ☐ No network call exists anywhere in the codebase outside the consent-gated egress module; proven by grep and by architecture test.
- ☐ The four data classes are annotated on every struct field that is logged, exported or transmitted.
- ☐ The client-facing PDF generator produces an accurate document from real consent state.
- ☐ Privacy policy and DPA drafted and reviewed by a qualified lawyer before the first paying customer.
- ☐ The forbidden-features list is in `CLAUDE.md` and in the agent instructions.
- ☐ The poisoned fixture test has been shown to fail when redaction is deliberately disabled.

10. Claude Code prompt

Act as `SEC` (Security & Privacy Lead) with `CTO` reviewing. Implement `24-PRIVACY-CONSENT-AND-LEGAL.md`: the frozen `ProjectConsent` contract, the egress gate that is the single network chokepoint, data-class annotations on every serialisable field, embedding encryption via OS keychain, project-scoped deletion, and the consent-expiry logic.

Constraints: default deny in every path; no `impl Default` that grants anything; no network call may exist outside the gate; embeddings must be physically incapable of serialisation into telemetry.

Then run the poisoned fixture test in section 8 and paste the grep output across all logs, telemetry payloads and the run bundle. Then deliberately disable the redaction filter, show the test failing, and re-enable it. If you find any existing code path that reaches the network without passing the gate, stop and report it as a **SEC** finding before writing anything else.